



CL-2001 Data Structures Lab # 14

Objectives:

- Graphs
- Hashing

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. There must be a block of comments at start of every question's code by students; the block should contain brief description about functionality of code.
2. Comment on every function about its functionality.
3. Use understandable name of variables.
4. Proper indentation of code is essential
5. Write a C++ statement(s) for each of the following task one after the other, in the same order.
6. Make a Microsoft Word file and paste all of your C++ code with all possible screenshots of every **task output in MS word and submit .cpp file with word file.**
7. Make separate .cpp files for all tasks and use this format **22F-1234_Task1.cpp.**
8. First think about statement problems and then write/draw your logic on copy.
9. After copy pencil work, code the problem statement on MS Studio C++ compiler.
10. At the end when you done your tasks, attached C++ created files in MS word file and make your submission on Google classroom. (Make sure your submission is completed).
11. Please submit your file in this format **22F-1234_L1.**
12. Do not submit your assignment **after the deadline.**
13. **Do not copy code from any source otherwise you will be penalized with negative marks.**

Problem 1 | Implement the following hashing techniques in C++.

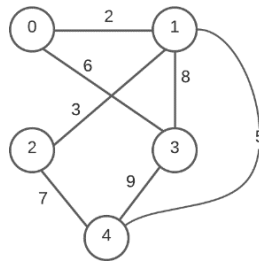
- Linear Probing
- Quadratic Probing

The following functions for all the algorithms should be implemented.

- Insert
- Delete
- Search

Problem 2 | Dijkstra Algorithm

Implement Dijkstra algorithm. Use the driver code given below.



Consider the following graph.

```
int graph[V][V] = { {0, 2, 0, 6, 0},
                    {2, 0, 3, 8, 5},
                    {0, 3, 0, 0, 7},
                    {6, 8, 0, 0, 9},
                    {0, 5, 7, 9, 0} };

void printSolution(int dist[]) {
    std::cout << "Vertex    Distance from Source\n";
    for (int i = 0; i < V; i++)
        std::cout << i << "        " << dist[i] << "\n";
}

int minDistance(int dist[], bool sptSet[]) {
    return ;
}

void dijkstra(int graph[V][V], int src) {
}
```

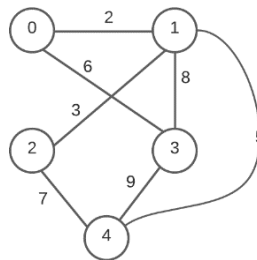
Output:

Microsoft Visual Studio Debug Console

Vertex	Distance from Source
0	0
1	2
2	5
3	6
4	7

Problem 3 | Prims Algorithm

Implement Prims algorithm. Use the driver code given below.



Consider the following graph.

```

int graph[V][V] = { {0, 2, 0, 6, 0},
                    {2, 0, 3, 8, 5},
                    {0, 3, 0, 0, 7},
                    {6, 8, 0, 0, 9},
                    {0, 5, 7, 9, 0} };

void printMST(int parent[], int graph[V][V]) {
    std::cout << "Edge  Weight\n";
    for (int i = 1; i < V; i++)
        std::cout << parent[i] << " - " << i << "      " << graph[i][parent[i]] << "\n";
}

int minKey(int key[], bool mstSet[]) {
    return ;
}

void primMST(int graph[V][V]) {
}

```

Output:



Microsoft Visual Studio

Edge	Weight
0 - 1	2
1 - 2	3
0 - 3	6
1 - 4	5

😊 Best of luck